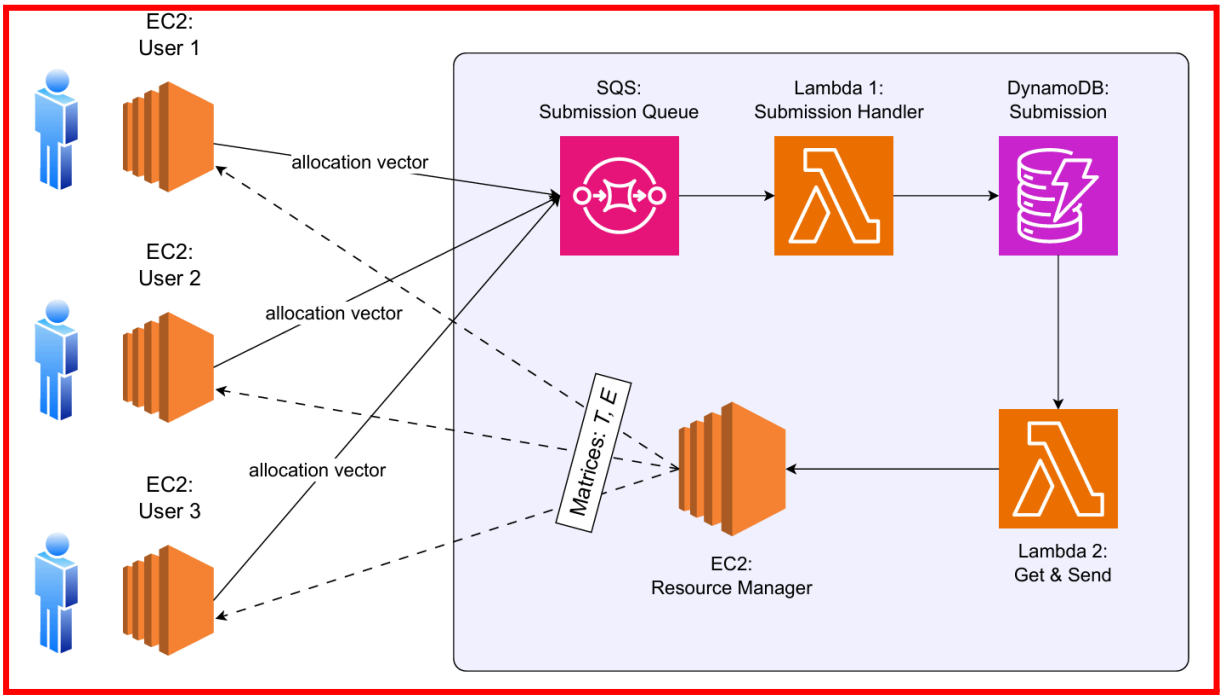
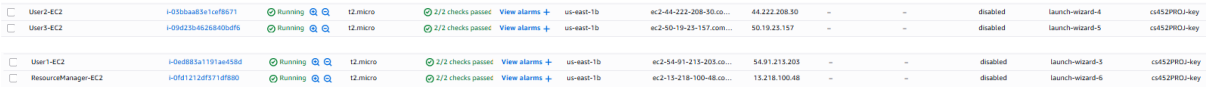
HY452 - Assignment 3 - CSD4647
ΙΩΑΝΝΗΣ ΚΑΣΤΡΙΝΑΚΗΣ

The overall architecture is divided in two parts: one part relates to the implementation of each user and the other to the implementation of the cloud provider.
The logic of its user will be placed into an EC2. An EC2 is a virtual machine. So in this virtual machine you will create a python flask application which will be responsible for 2 things (1) listen for http requests by the Resource Manager (2) being able to send its decision.
The actual brain of the Cloud Provider is the EC2 component called Resource Manager.
The other components inside the Cloud Provider (sqs, lambdas, dynamodb) are pre-processing steps.

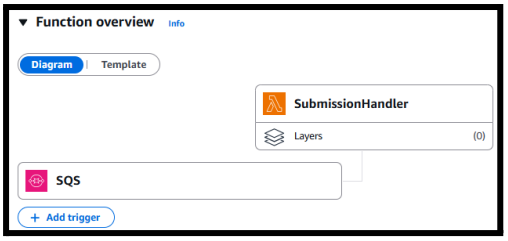


EC2: (3 users and 1 for resource manager)

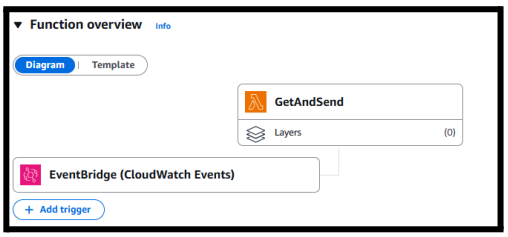


User2-EC2	i-03bbaa83e1cef8671	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-44-222-208-30.co...	44.222.208.30	–	–	disabled	launch-wizard-4	cs452PROJ-key
User3-EC2	i-09d23b4626840bdf6	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-50-19-23-157.com...	50.19.23.157	–	–	disabled	launch-wizard-5	cs452PROJ-key
User1-EC2	i-0ed883a11914e458d	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-54-91-213-203.co...	54.91.213.203	–	–	disabled	launch-wizard-3	cs452PROJ-key
ResourceManager-EC2	i-0fd1212df371df880	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-13-218-100-48.co...	13.218.100.48	–	–	disabled	launch-wizard-6	cs452PROJ-key

Lambda 1: Submission Handler



Lambda 2: Get &Send



SQS: Submission Queue

SubmissionQueue	Standard	2026-01-19T19:53+02:00	0	0	Amazon SQS key (SSE-SQS)	-

DynamoDB: Submission (after running step1 and step2)

Table: SubmissionTable - Items returned (6)

Scan started on January 21, 2026, 21:19:33

Actions ▼ Create item

1

user_id...	timesta...	allocation_vector	status	step
3	1769023159	[{"N":"1"},{"N":"1"},{"N":"1"},{"N":"1"},{"N":"0"}]	processed	2
3	1769023139	[{"N":"1"},{"N":"1"},{"N":"1"},{"N":"0"},{"N":"1"}]	processed	1
2	1769023160	[{"N":"1"},{"N":"0"},{"N":"0"},{"N":"1"},{"N":"1"}]	processed	2
2	1769023138	[{"N":"0"},{"N":"1"},{"N":"1"},{"N":"1"},{"N":"0"}]	processed	1
1	1769023159	[{"N":"0"},{"N":"1"},{"N":"1"},{"N":"0"},{"N":"0"}]	processed	2
1	1769023138	[{"N":"0"},{"N":"0"},{"N":"0"},{"N":"1"},{"N":"1"}]	processed	1

Everything is running on port 5000

A fully working example:

IMPORTANT NOTE : in the lambda function Get&Send we need to change every time the resource manager public ip (in the top of the file). For user i use private ip so we are okay

STEP1

First we start the Flask applications (in every EC2 terminal),
then connect again from another ec2 user terminal (lets say user1)
from there we run the commands:
curl http://ip-172-31-23-96.ec2.internal:5000/calculate
curl http://ip-172-31-26-36.ec2.internal:5000/calculate
curl http://ip-172-31-20-134.ec2.internal:5000/calculate

in the users we take:

```
============================================================
[User 1] STEP 1: Starting optimization calculation...
============================================================
Using ORIGINAL execution times: [6.  5.  4.  3.5 3. ]
[User 1] Step 1 optimal allocation found:
  Allocation vector: [0, 0, 0, 1, 1]
  Expected utility: 0.122489
  Expected max time: 3.50s
  Expected expense: 12.30€
[User 1] ✓ Step 1 allocation sent to SQS
============================================================

============================================================
[User 2] STEP 1: Starting optimization calculation...
============================================================
Using ORIGINAL execution times: [5.  4.2 3.6 3.  2.8]
```

[User 2] Step 1 optimal allocation found:
  Allocation vector: [0, 1, 1, 1, 0]
  Expected utility: 0.097534
  Expected max time: 4.20s
  Expected expense: 15.84€
[User 2] ✓ Step 1 allocation sent to SQS
==========

==========
[User 3] STEP 1: Starting optimization calculation...
==========
Using ORIGINAL execution times: [4. 3.5 3.2 2.8 2.4]
[User 3] Step 1 optimal allocation found:
  Allocation vector: [1, 1, 1, 0, 1]
  Expected utility: 0.090596
  Expected max time: 4.00s
  Expected expense: 17.80€
[User 3] ✓ Step 1 allocation sent to SQS
==========
(of course from 3 different terminals/ec2 instances)

in the dynamoDB we have :

Table: SubmissionTable - Items returned (3)

Scan started on January 21, 2026, 22:31:56

user_id...	timesta...	allocation_vector	status	step
2	1769027509	[{ "N" : "0" }, { "N" : "1...	pending	1
1	1769027509	[{ "N" : "0" }, { "N" : "0...	pending	1
3	1769027509	[{ "N" : "1" }, { "N" : "1...	pending	1

AFTER THE GET AND SENT IS TRIGGERED WE CAN SEE THAT:

in the users we take:
==========
[User 1] STEP 1: Received matrices from Resource Manager
==========
[User 1] STEP 1 Actual results (with resource sharing):
  Allocation used: [0, 0, 0, 1, 1]
  Resources used: [3, 4]
  Actual max time: 7.00s
  Actual expense: 12.30€
  Actual utility: 0.101947

[User 1] ✓ Received updated execution times for STEP 2:
  Original T̂: [6. 5. 4. 3.5 3.]

```
  Updated T̂_new: [6. 5. 4. 7. 6.]
  Difference:     [0.  0.  0.  3.5 3. ]

[User 1] Ready for Step 2! Call /calculate_step2
================================================================

================================================================
[User 2] STEP 1: Received matrices from Resource Manager
================================================================
[User 2] STEP 1 Actual results (with resource sharing):
  Allocation used: [0, 1, 1, 1, 0]
  Resources used: [1, 2, 3]
  Actual max time: 8.40s
  Actual expense: 15.84€
  Actual utility: 0.081508

[User 2] ✓ Received updated execution times for STEP 2:
  Original T̂:     [5.  4.2 3.6 3.  2.8]
  Updated T̂_new: [5.  8.4 7.2 6.  2.8]
  Difference:     [0.  4.2 3.6 3.  0. ]

[User 2] Ready for Step 2! Call /calculate_step2
================================================================

================================================================
[User 3] STEP 1: Received matrices from Resource Manager
================================================================
[User 3] STEP 1 Actual results (with resource sharing):
  Allocation used: [1, 1, 1, 0, 1]
  Resources used: [0, 1, 2, 4]
  Actual max time: 7.00s
  Actual expense: 17.80€
  Actual utility: 0.079949

[User 3] ✓ Received updated execution times for STEP 2:
  Original T̂:     [4.  3.5 3.2 2.8 2.4]
  Updated T̂_new: [8.  7.  6.4 2.8 4.8]
  Difference:     [4.  3.5 3.2 0.  2.4]

[User 3] Ready for Step 2! Call /calculate_step2
================================================================
```

in the resource manager we take:

```
================================================================
STEP 1: RECEIVED ALLOCATIONS FROM LAMBDA 2
================================================================
```

```
Received 3 allocations:
  User 1: [0, 0, 0, 1, 1]
  User 2: [0, 1, 1, 1, 0]
  User 3: [1, 1, 1, 0, 1]

============================================================
CALCULATING T AND E MATRICES
============================================================

Allocation matrix a:
[[0 0 0 1 1]
 [0 1 1 1 0]
 [1 1 1 0 1]]

Resource usage counts n: [1 2 2 2 2]
(This shows resource multiplexing/conflicts)

T matrix (with multiplexing):
[[0.  0.  0.  7.  6. ]
 [0.  8.4 7.2 6.  0. ]
 [4.  7.  6.4 0.  4.8]]

E matrix (expenses):
[[0.   0.   0.   6.3  6.  ]
 [0.   5.04 5.4  5.4  0.  ]
 [4.   4.2  4.8  0.   4.8 ]]
============================================================


============================================================
CALCULATING T-HAT-NEW FOR STEP 2 (Section 2.2)
============================================================
Formula: t-hat-new[i][j] = t-hat[i][j] + sum(t[i][j]/n)

Original T-hat:
[[6.  5.  4.  3.5 3. ]
 [5.  4.2 3.6 3.  2.8]
 [4.  3.5 3.2 2.8 2.4]]

Updated T-hat-new (incorporating Step 1 experience):
[[6.  5.  4.  7.  6. ]
 [5.  8.4 7.2 6.  2.8]
 [8.  7.  6.4 2.8 4.8]]

Difference (what users learned from Step 1):
[[0.  0.  0.  3.5 3. ]
 [0.  4.2 3.6 3.  0. ]
 [4.  3.5 3.2 0.  2.4]]
```

==========

==========
SENDING RESULTS TO USERS
==========

Sending to User 1 at http://ip-172-31-23-96.ec2.internal:5000/update
User 1 acknowledged (Step 1)
  Actual utility: 0.10194719135487816

Sending to User 2 at http://ip-172-31-26-36.ec2.internal:5000/update
User 2 acknowledged (Step 1)
  Actual utility: 0.08150756390193009

Sending to User 3 at http://ip-172-31-20-134.ec2.internal:5000/update
User 3 acknowledged (Step 1)
  Actual utility: 0.07994883274704188

==========
STEP 1 PROCESS COMPLETE
==========

in the dynamoDB we have :

Table: SubmissionTable - Items returned (3)

Scan started on January 21, 2026, 22:22:10

user_id...	timesta...	allocation_vector	status	step
2	1769026547	[{ "N" : "0" }, { "N" : "1...	processed	1
1	1769026547	[{ "N" : "0" }, { "N" : "0...	processed	1
3	1769026548	[{ "N" : "1" }, { "N" : "1...	processed	1

STEP2

in the user 1 terminal we do:
curl http://ip-172-31-23-96.ec2.internal:5000/calculate_step2
curl http://ip-172-31-26-36.ec2.internal:5000/calculate_step2
curl http://ip-172-31-20-134.ec2.internal:5000/calculate_step2

in the users we take:

==========
[User 1] STEP 2: Recalculating with UPDATED execution times...
==========
Original $\hat{T}$: [6. 5. 4. 3.5 3.]
Updated $\hat{T}$_new: [6. 5. 4. 7. 6.]
Change: [0. 0. 0. 3.5 3.]

```
[User 1] Step 2 optimal allocation found:
  Allocation vector: [0, 1, 1, 0, 0]
  Expected utility: 0.114811
  Expected max time: 5.00s
  Expected expense: 12.00€

  COMPARISON WITH STEP 1:
  Step 1 allocation: [0, 0, 0, 1, 1]
  Step 2 allocation: [0, 1, 1, 0, 0]
  Changed? True
[User 1] ✓ Step 2 allocation sent to SQS
============================================================

============================================================
[User 2] STEP 2: Recalculating with UPDATED execution times...
============================================================
Original T̂:  [5.  4.2 3.6 3.  2.8]
Updated T̂_new: [5.  8.4 7.2 6.  2.8]
Change:      [0.  4.2 3.6 3.  0. ]

[User 2] Step 2 optimal allocation found:
  Allocation vector: [1, 0, 0, 1, 1]
  Expected utility: 0.071388
  Expected max time: 6.00s
  Expected expense: 21.40€

  COMPARISON WITH STEP 1:
  Step 1 allocation: [0, 1, 1, 1, 0]
  Step 2 allocation: [1, 0, 0, 1, 1]
  Changed? True
[User 2] ✓ Step 2 allocation sent to SQS
============================================================

============================================================
[User 3] STEP 2: Recalculating with UPDATED execution times...
============================================================
Original T̂:  [4.  3.5 3.2 2.8 2.4]
Updated T̂_new: [8.  7.  6.4 2.8 4.8]
Change:      [4.  3.5 3.2 0.  2.4]

[User 3] Step 2 optimal allocation found:
  Allocation vector: [1, 1, 1, 1, 0]
  Expected utility: 0.050632
  Expected max time: 8.00s
  Expected expense: 31.04€

  COMPARISON WITH STEP 1:
  Step 1 allocation: [1, 1, 1, 0, 1]
```

Step 2 allocation: [1, 1, 1, 1, 0]
Changed? True
[User 3] ✓ Step 2 allocation sent to SQS
==========

in the dynamoDB we have :

Table: SubmissionTable - Items returned (6)

Scan started on January 21, 2026, 22:34:53

user_id...	timesta...	allocation_vector	status	step
2	1769027509	[{"N":"0"},{"N":"1...	processed	1
2	1769027689	[{"N":"1"},{"N":"0...	pending	2
1	1769027509	[{"N":"0"},{"N":"0...	processed	1
1	1769027688	[{"N":"0"},{"N":"1...	pending	2
3	1769027509	[{"N":"1"},{"N":"1...	processed	1
3	1769027689	[{"N":"1"},{"N":"1...	pending	2

AFTER THE GET AND SENT IS TRIGGERED WE CAN SEE THAT:

in the users we take:

==========
[User 1] STEP 2: Received matrices from Resource Manager
==========
[User 1] STEP 2 Actual results (with resource sharing):
Allocation used: [0, 1, 1, 0, 0]
Resources used: [1, 2]
Actual max time: 10.00s
Actual expense: 12.00€
Actual utility: 0.090416
==========

==========
[User 2] STEP 2: Received matrices from Resource Manager
==========
[User 2] STEP 2 Actual results (with resource sharing):
Allocation used: [1, 0, 0, 1, 1]
Resources used: [0, 3, 4]
Actual max time: 10.00s
Actual expense: 16.00€
Actual utility: 0.076220
==========

==========
[User 3] STEP 2: Received matrices from Resource Manager
==========
[User 3] STEP 2 Actual results (with resource sharing):

```
  Allocation used: [1, 1, 1, 1, 0]
  Resources used: [0, 1, 2, 3]
  Actual max time: 8.00s
  Actual expense: 18.04€
  Actual utility: 0.076217
============================================================
```

in the resource manager we take:

```
============================================================
STEP 2: RECEIVED ALLOCATIONS FROM LAMBDA 2
============================================================

Received 3 allocations:
  User 1: [0, 1, 1, 0, 0]
  User 2: [1, 0, 0, 1, 1]
  User 3: [1, 1, 1, 1, 0]

============================================================
CALCULATING T AND E MATRICES
============================================================

Allocation matrix a:
[[0 1 1 0 0]
 [1 0 0 1 1]
 [1 1 1 1 0]]

Resource usage counts n: [2 2 2 2 1]
(This shows resource multiplexing/conflicts)

T matrix (with multiplexing):
[[ 0.  10.   8.   0.   0. ]
 [10.   0.   0.   6.   2.8]
 [ 8.   7.   6.4  5.6  0. ]]

E matrix (expenses):
[[0.   6.   6.   0.   0.  ]
 [5.   0.   0.   5.4  5.6 ]
 [4.   4.2  4.8  5.04 0.  ]]
============================================================


============================================================
CALCULATING T-HAT-NEW FOR STEP 2 (Section 2.2)
============================================================
Formula: t-hat-new[i][j] = t-hat[i][j] + sum(t[i][j]/n)

Original T-hat:
```

```
[[6.  5.  4.  3.5 3. ]
 [5.  4.2 3.6 3.  2.8]
 [4.  3.5 3.2 2.8 2.4]]

Updated T-hat-new (incorporating Step 1 experience):
[[ 6.  10.   8.   3.5  3. ]
 [10.   4.2  3.6  6.   5.6]
 [ 8.   7.   6.4  5.6  2.4]]

Difference (what users learned from Step 1):
[[0.  5.  4.  0.  0. ]
 [5.  0.  0.  3.  2.8]
 [4.  3.5 3.2 2.8 0. ]]
============================================================


============================================================
SENDING RESULTS TO USERS
============================================================

Sending to User 1 at http://ip-172-31-23-96.ec2.internal:5000/update
User 1 acknowledged (Step 2)
  Actual utility: 0.09041591320072334

Sending to User 2 at http://ip-172-31-26-36.ec2.internal:5000/update
User 2 acknowledged (Step 2)
  Actual utility: 0.07621951219512195

Sending to User 3 at http://ip-172-31-20-134.ec2.internal:5000/update
User 3 acknowledged (Step 2)
  Actual utility: 0.0762171885003506

============================================================
STEP 2 PROCESS COMPLETE
============================================================
```

in the dynamoDB we have :

Table: SubmissionTable - Items returned (6)

Scan started on January 21, 2026, 22:35:55

user_id...	timesta...	allocation_vector	status	step
2	1769027509	[{"N":"0"},{"N":"1...	processed	1
2	1769027689	[{"N":"1"},{"N":"0...	processed	2
1	1769027509	[{"N":"0"},{"N":"0...	processed	1
1	1769027688	[{"N":"0"},{"N":"1...	processed	2
3	1769027509	[{"N":"1"},{"N":"1...	processed	1
3	1769027689	[{"N":"1"},{"N":"1...	processed	2

STEP 1 results Example

User (task)	Allocation vector	Expected Utility	Actual Utility
user1	(0, 0, 0, 1, 1)	0.1225	0.1019
user2	(0, 1, 1, 1, 0)	0.0975	0.0815
user3	(1, 1, 1, 0, 1)	0.0906	0.0799

STEP 2 results Example

User (task)	Allocation vector	Expected Utility	Actual Utility
user1	(0, 1, 1, 0, 0)	0.1148	0.0904
user2	(1, 0, 0, 1, 1)	0.0714	0.0762
user3	(1, 1, 1, 1, 0)	0.0506	0.0762

CODES

lambda1:

```
import json
import boto3
import time

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('SubmissionTable')

def lambda_handler(event, context):
    #triggered by sqs messages
    #extracts allocation vector and stores in dynamodb
    #saves step field from message

    print(f"Received {len(event['Records'])} messages")
```

```
    for record in event['Records']:
        #parse message body
        body = json.loads(record['body'])

        user_id = body['user_id']
        allocation = body['allocation_vector']
        step = body.get('step', 1)

        #store in dynamodb
        table.put_item(
            Item={
                'user_id': str(user_id),
                'timestamp': int(time.time()),
                'allocation_vector': allocation,
                'status': 'pending',
                'step': step
            }
        )

        print(f"Stored Step {step} allocation for User {user_id}:
{allocation}")

    return {
        'statusCode': 200,
        'body': json.dumps('Successfully processed messages')
    }
```

lambda2:

```
import json
import boto3
import urllib3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('SubmissionTable')
http = urllib3.PoolManager()

#resource manager endpoint - using public ip
#update this ip if resource manager restarts
RESOURCE_MANAGER_URL = 'http://3.85.63.182:5000/process'

def lambda_handler(event, context):
```

```
    #retrieves all allocations from dynamodb and sends to resource
manager
    #handles both step 1 and step 2 submissions

    print("="*60)
    print("Lambda 2: Get & Send - Starting")
    print("="*60)

    #scan dynamodb for all pending submissions
    response = table.scan(
        FilterExpression='#status = :pending',
        ExpressionAttributeNames={'#status': 'status'},
        ExpressionAttributeValues={':pending': 'pending'}
    )

    items = response['Items']
    print(f"Found {len(items)} pending submissions")

    #check if all 3 users have submitted
    if len(items) < 3:
        print(f"Only {len(items)}/3 users submitted. Waiting for
more...")
        return {
            'statusCode': 200,
            'body': json.dumps(f'Waiting for all users. Currently:
{len(items)}/3')
        }

    #sort by user_id to ensure correct order
    items_sorted = sorted(items, key=lambda x: int(x['user_id']))

    #detect which step and convert decimal to int
    step = int(items_sorted[0].get('step', 1))
    print(f"Processing STEP {step} submissions")

    #extract allocations and convert decimal to int
    allocations = {}
    for item in items_sorted:
        user_id = item['user_id']
        #convert decimal values to int
        allocation = [int(x) for x in item['allocation_vector']]
        allocations[user_id] = allocation
        print(f"User {user_id}: {allocation}")
```

```python
#prepare payload for resource manager
payload = {
    'allocations': allocations,
    'user_count': len(allocations),
    'step': step
}

print(f"\n{'='*60}")
print(f"Sending STEP {step} to Resource Manager...")
print(f"{'='*60}")

#send to resource manager
try:
    encoded_data = json.dumps(payload).encode('utf-8')

    response = http.request(
        'POST',
        RESOURCE_MANAGER_URL,
        body=encoded_data,
        headers={'Content-Type': 'application/json'},
        timeout=30.0
    )

    print(f"Resource Manager response: {response.status}")
    print(f"Response body: {response.data.decode('utf-8')}")

    #update dynamodb items to processed
    for item in items_sorted:
        table.update_item(
            Key={
                'user_id': item['user_id'],
                'timestamp': item['timestamp']
            },
            UpdateExpression='SET #status = :processed',
            ExpressionAttributeNames={'#status': 'status'},
            ExpressionAttributeValues={':processed': 'processed'}
        )

    print(f"Updated all STEP {step} items to processed")

    return {
        'statusCode': 200,
```

```
            'body': json.dumps({
                'message': f'Successfully sent Step {step} to Resource
Manager',
                'step': step,
                'allocations': allocations
            })
        }

    except Exception as e:
        print(f"Error sending to Resource Manager: {str(e)}")
        return {
            'statusCode': 500,
            'body': json.dumps({'error': str(e)})
        }
```

resource_manager:

```
from flask import Flask, jsonify, request
import numpy as np
import requests
import json

app = Flask(__name__)

#provider data from assignment section 1.4
PRICES = np.array([1, 1.2, 1.5, 1.8, 2])

#execution time matrix t-hat from assignment
T_HAT = np.array([
    [6,   5,   4,   3.5, 3  ],    #user 1
    [5,   4.2, 3.6, 3,   2.8],    #user 2
    [4,   3.5, 3.2, 2.8, 2.4]     #user 3
])

#user endpoints using private dns
USER_ENDPOINTS = {
    1: 'http://ip-172-31-23-96.ec2.internal:5000/update',
    2: 'http://ip-172-31-26-36.ec2.internal:5000/update',
    3: 'http://ip-172-31-20-134.ec2.internal:5000/update'
}

#track which step we are on
CURRENT_STEP = 1
```

```
def calculate_T_and_E_matrices(allocations):
    #calculate actual t and e matrices considering resource
multiplexing
    #formula from assignment section 2.1:
    #t[i][j] = n[j] * t-hat[i][j] where n[j] = number of users on
resource j
    #e[i][j] = a[i][j] * t-hat[i][j] * p[j]

    print("\n" + "="*60)
    print("CALCULATING T AND E MATRICES")
    print("="*60)

    #convert allocations to numpy array
    a = np.zeros((3, 5), dtype=int)
    for user_id_str, allocation in allocations.items():
        user_idx = int(user_id_str) - 1
        a[user_idx] = np.array(allocation)

    print("\nAllocation matrix a:")
    print(a)

    #calculate resource usage: n[j] = how many users use resource j
    n = np.sum(a, axis=0)
    print(f"\nResource usage counts n: {n}")
    print("(This shows resource multiplexing/conflicts)")

    #initialize t and e matrices
    T_matrix = np.zeros((3, 5))
    E_matrix = np.zeros((3, 5))

    #calculate t matrix (time with multiplexing effect)
    for i in range(3):
        for j in range(5):
            if a[i][j] > 0:
                T_matrix[i][j] = n[j] * T_HAT[i][j]
            else:
                T_matrix[i][j] = 0

    #calculate e matrix (expenses)
    for i in range(3):
        for j in range(5):
            if a[i][j] > 0:
```

```
                E_matrix[i][j] = a[i][j] * T_HAT[i][j] * PRICES[j]
            else:
                E_matrix[i][j] = 0

    print("\nT matrix (with multiplexing):")
    print(T_matrix)
    print("\nE matrix (expenses):")
    print(E_matrix)
    print("="*60 + "\n")

    return T_matrix, E_matrix, n


def calculate_T_HAT_NEW(T_matrix, n):
    #calculate updated execution time matrix for step 2
    #formula from assignment section 2.2:
    #t-hat-new[i][j] = t-hat[i][j] + sum(t[i][j]/n)
    #where:
    #t-hat[i][j] = original execution time
    #t[i][j] = actual execution time from step 1 (with multiplexing)
    #n = number of users per resource

    print("\n" + "="*60)
    print("CALCULATING T-HAT-NEW FOR STEP 2 (Section 2.2)")
    print("="*60)
    print("Formula: t-hat-new[i][j] = t-hat[i][j] + sum(t[i][j]/n)")

    T_HAT_NEW = np.copy(T_HAT)

    #for each user
    for i in range(3):
        #for each resource
        for j in range(5):
            #if resource j was used by anyone
            if n[j] > 0:
                #calculate the sum term
                #this represents the average additional time due to
sharing
                sum_term = T_matrix[i][j] / n[j] if T_matrix[i][j] > 0
else 0

                #update: new estimate = old estimate + learned delay
                T_HAT_NEW[i][j] = T_HAT[i][j] + sum_term
```

```
    print("\nOriginal T-hat:")
    print(T_HAT)
    print("\nUpdated T-hat-new (incorporating Step 1 experience):")
    print(T_HAT_NEW)
    print("\nDifference (what users learned from Step 1):")
    print(T_HAT_NEW - T_HAT)
    print("="*60 + "\n")

    return T_HAT_NEW


@app.route('/')
def home():
    #health check
    return jsonify({
        'status': 'running',
        'service': 'Resource Manager',
        'message': 'CS452 Cloud Provider - Ready to process
allocations',
        'current_step': CURRENT_STEP
    })


@app.route('/process', methods=['POST'])
def process_allocations():
    #receive allocations from lambda 2
    #calculate t and e matrices
    #send results back to all users
    global CURRENT_STEP

    data = request.json
    allocations = data['allocations']
    step = data.get('step', 1)

    print("\n" + "="*60)
    print(f"STEP {step}: RECEIVED ALLOCATIONS FROM LAMBDA 2")
    print("="*60)

    print(f"\nReceived {len(allocations)} allocations:")
    for user_id, allocation in allocations.items():
        print(f"  User {user_id}: {allocation}")
```

```
    #calculate t and e matrices (and n for step 2 calculation)
    T_matrix, E_matrix, n = calculate_T_and_E_matrices(allocations)

    #calculate t-hat-new for step 2
    T_HAT_NEW = calculate_T_HAT_NEW(T_matrix, n)

    #prepare response payload including t-hat-new
    payload = {
        'T_matrix': T_matrix.tolist(),
        'E_matrix': E_matrix.tolist(),
        'T_HAT_NEW': T_HAT_NEW.tolist(),
        'step': step
    }

    #send to all users
    print("\n" + "="*60)
    print("SENDING RESULTS TO USERS")
    print("="*60)

    results = {}
    for user_id, endpoint in USER_ENDPOINTS.items():
        try:
            print(f"\nSending to User {user_id} at {endpoint}")
            response = requests.post(
                endpoint,
                json=payload,
                timeout=10
            )

            if response.status_code == 200:
                user_response = response.json()
                print(f"User {user_id} acknowledged (Step {step})")
                print(f"  Actual utility:
{user_response.get('actual_utility', 'N/A')}")
                results[user_id] = 'success'
            else:
                print(f"User {user_id} error: {response.status_code}")
                results[user_id] = 'failed'

        except Exception as e:
            print(f"User {user_id} connection error: {str(e)}")
            results[user_id] = 'error'
```

```
    print("\n" + "="*60)
    print(f"STEP {step} PROCESS COMPLETE")
    print("="*60 + "\n")

    CURRENT_STEP = step + 1

    return jsonify({
        'status': 'success',
        'step': step,
        'message': f'Processed Step {step} allocations and notified
users',
        'T_matrix': T_matrix.tolist(),
        'E_matrix': E_matrix.tolist(),
        'T_HAT_NEW': T_HAT_NEW.tolist(),
        'notification_results': results
    })


if __name__ == '__main__':
    print("\n" + "="*60)
    print("RESOURCE MANAGER STARTING")
    print("="*60)
    print("\nProvider Configuration:")
    print(f"  Prices: {PRICES}")
    print(f"\n  Execution Time Matrix T-hat:")
    print(T_HAT)
    print(f"\n  User endpoints configured: {len(USER_ENDPOINTS)}")
    for uid, endpoint in USER_ENDPOINTS.items():
        print(f"    User {uid}: {endpoint}")
    print("="*60 + "\n")

    app.run(host='0.0.0.0', port=5000, debug=True)
```

user1_app:

```
from flask import Flask, jsonify, request
import numpy as np
from itertools import product
import boto3
import json

app = Flask(__name__)

#user 1 parameters (student id: 47)
```

```
USER_ID = 1
NUM_SUBTASKS = 2
W_TIME = 0.47
W_EXPENSE = 0.53
MAX_TIME = 500
MAX_EXPENSE = 20

PRICES = np.array([1, 1.2, 1.5, 1.8, 2])
EXEC_TIMES = np.array([6, 5, 4, 3.5, 3])
NUM_RESOURCES = 5

#sqs configuration
SQS_QUEUE_URL =
'https://sqs.us-east-1.amazonaws.com/519661830216/SubmissionQueue'
sqs = boto3.client('sqs', region_name='us-east-1')

#step 2 tracking variables
STEP2_EXEC_TIMES = None
CURRENT_STEP = 1
STEP1_EXPECTED_UTILITY = None
STEP1_ACTUAL_UTILITY = None
STEP1_ALLOCATION = None
STEP2_ALLOCATION = None


def calculate_utility(allocation, exec_times, prices, w_t, w_e):
    #calculate utility for given allocation
    times_for_allocated = [exec_times[j] for j in range(NUM_RESOURCES)
                           if allocation[j] > 0]
    if not times_for_allocated:
        return 0

    max_time = max(times_for_allocated)
    total_expense = sum(allocation[j] * exec_times[j] * prices[j]
                        for j in range(NUM_RESOURCES))

    denominator = w_t * max_time + w_e * total_expense
    return 1 / denominator if denominator > 0 else 0


def find_optimal_allocation():
    #find optimal allocation using original execution times (step 1)
    best_utility = 0
```

```
    best_allocation = None
    best_details = None

    for resources in product(range(NUM_RESOURCES),
repeat=NUM_SUBTASKS):
        if len(set(resources)) != NUM_SUBTASKS:
            continue

        allocation = np.zeros(NUM_RESOURCES, dtype=int)
        for r in resources:
            allocation[r] += 1

        times = [EXEC_TIMES[j] for j in range(NUM_RESOURCES)
                if allocation[j] > 0]
        max_time = max(times)
        total_expense = sum(allocation[j] * EXEC_TIMES[j] * PRICES[j]
                            for j in range(NUM_RESOURCES))

        if max_time > MAX_TIME or total_expense > MAX_EXPENSE:
            continue

        utility = calculate_utility(allocation, EXEC_TIMES, PRICES,
                                    W_TIME, W_EXPENSE)

        if utility > best_utility:
            best_utility = utility
            best_allocation = allocation
            best_details = {
                'max_time': max_time,
                'total_expense': total_expense,
                'utility': utility
            }

    return best_allocation, best_details


def find_optimal_allocation_step2(exec_times_step2):
    #find optimal allocation using updated execution times from step 2
    best_utility = 0
    best_allocation = None
    best_details = None
```

```
    for resources in product(range(NUM_RESOURCES),
repeat=NUM_SUBTASKS):
        if len(set(resources)) != NUM_SUBTASKS:
            continue

        allocation = np.zeros(NUM_RESOURCES, dtype=int)
        for r in resources:
            allocation[r] += 1

        times = [exec_times_step2[j] for j in range(NUM_RESOURCES)
                if allocation[j] > 0]
        max_time = max(times)
        total_expense = sum(allocation[j] * exec_times_step2[j] *
PRICES[j]
                            for j in range(NUM_RESOURCES))

        if max_time > MAX_TIME or total_expense > MAX_EXPENSE:
            continue

        utility = calculate_utility(allocation, exec_times_step2,
PRICES,
                                    W_TIME, W_EXPENSE)

        if utility > best_utility:
            best_utility = utility
            best_allocation = allocation
            best_details = {
                'max_time': max_time,
                'total_expense': total_expense,
                'utility': utility
            }

    return best_allocation, best_details


@app.route('/')
def home():
    #health check endpoint
    return jsonify({
        'status': 'running',
        'user_id': USER_ID,
        'message': 'User 1 Flask App - CS452 Project',
        'current_step': CURRENT_STEP,
```

```
        'step2_ready': STEP2_EXEC_TIMES is not None
    })


@app.route('/calculate', methods=['GET', 'POST'])
def calculate_allocation():
    #step 1: calculate optimal allocation using original t-hat
    global STEP1_EXPECTED_UTILITY, STEP1_ALLOCATION, CURRENT_STEP

    CURRENT_STEP = 1

    print(f"\n{'='*60}")
    print(f"[User {USER_ID}] STEP 1: Starting optimization
calculation...")
    print(f"{'='*60}")
    print(f"Using ORIGINAL execution times: {EXEC_TIMES}")

    allocation, details = find_optimal_allocation()

    if allocation is None:
        print(f"[User {USER_ID}] ERROR: No feasible allocation found!")
        return jsonify({'error': 'No feasible allocation found'}), 400

    STEP1_EXPECTED_UTILITY = details['utility']
    STEP1_ALLOCATION = allocation

    print(f"[User {USER_ID}] Step 1 optimal allocation found:")
    print(f"  Allocation vector: {allocation.tolist()}")
    print(f"  Expected utility: {details['utility']:.6f}")
    print(f"  Expected max time: {details['max_time']:.2f}s")
    print(f"  Expected expense: {details['total_expense']:.2f}€")

    message = {
        'user_id': USER_ID,
        'step': 1,
        'allocation_vector': allocation.tolist(),
        'expected_utility': details['utility'],
        'expected_max_time': details['max_time'],
        'expected_expense': details['total_expense']
    }

    try:
        response = sqs.send_message(
```

```
            QueueUrl=SQS_QUEUE_URL,
            MessageBody=json.dumps(message)
        )
        print(f"[User {USER_ID}] Step 1 allocation sent to SQS")
        print(f"{'='*60}\n")

        return jsonify({
            'user_id': USER_ID,
            'step': 1,
            'allocation_vector': allocation.tolist(),
            'expected_utility': details['utility'],
            'expected_max_time': details['max_time'],
            'expected_expense': details['total_expense'],
            'message_id': response['MessageId'],
            'status': 'submitted_to_queue'
        })

    except Exception as e:
        print(f"[User {USER_ID}] Error sending to SQS: {e}")
        return jsonify({'error': str(e)}), 500


@app.route('/calculate_step2', methods=['GET', 'POST'])
def calculate_step2():
    #step 2: recalculate allocation using updated t-hat-new
    global CURRENT_STEP, STEP2_EXEC_TIMES, STEP2_ALLOCATION

    if STEP2_EXEC_TIMES is None:
        return jsonify({
            'error': 'Step 2 not ready. Must complete Step 1 first to
receive T-hat-new.'
        }), 400

    CURRENT_STEP = 2

    print(f"\n{'='*60}")
    print(f"[User {USER_ID}] STEP 2: Recalculating with UPDATED
execution times...")
    print(f"{'='*60}")
    print(f"Original T-hat:  {EXEC_TIMES}")
    print(f"Updated T-hat-new: {STEP2_EXEC_TIMES}")
    print(f"Change:      {STEP2_EXEC_TIMES - EXEC_TIMES}")
```

```
    allocation, details =
find_optimal_allocation_step2(STEP2_EXEC_TIMES)

    if allocation is None:
        print(f"[User {USER_ID}] ERROR: No feasible allocation found in
Step 2!")
        return jsonify({'error': 'No feasible allocation found'}), 400

    STEP2_ALLOCATION = allocation

    print(f"\n[User {USER_ID}] Step 2 optimal allocation found:")
    print(f"  Allocation vector: {allocation.tolist()}")
    print(f"  Expected utility: {details['utility']:.6f}")
    print(f"  Expected max time: {details['max_time']:.2f}s")
    print(f"  Expected expense: {details['total_expense']:.2f}€")

    if STEP1_ALLOCATION is not None:
        print(f"\n  COMPARISON WITH STEP 1:")
        print(f"  Step 1 allocation: {STEP1_ALLOCATION.tolist()}")
        print(f"  Step 2 allocation: {allocation.tolist()}")
        print(f"  Changed? {not np.array_equal(STEP1_ALLOCATION,
allocation)}")

    message = {
        'user_id': USER_ID,
        'step': 2,
        'allocation_vector': allocation.tolist(),
        'expected_utility': details['utility'],
        'expected_max_time': details['max_time'],
        'expected_expense': details['total_expense']
    }

    try:
        response = sqs.send_message(
            QueueUrl=SQS_QUEUE_URL,
            MessageBody=json.dumps(message)
        )
        print(f"[User {USER_ID}] Step 2 allocation sent to SQS")
        print(f"{'='*60}\n")

        return jsonify({
            'user_id': USER_ID,
            'step': 2,
```

```
            'allocation_vector': allocation.tolist(),
            'expected_utility': details['utility'],
            'expected_max_time': details['max_time'],
            'expected_expense': details['total_expense'],
            'message_id': response['MessageId'],
            'status': 'submitted_to_queue'
        })

    except Exception as e:
        print(f"[User {USER_ID}] Error sending to SQS: {e}")
        return jsonify({'error': str(e)}), 500


@app.route('/update', methods=['POST'])
def receive_update():
    #receive updated t and e matrices from resource manager
    #uses stored allocations instead of recalculating
    global STEP2_EXEC_TIMES, STEP1_ACTUAL_UTILITY

    data = request.json
    step = data.get('step', 1)

    print(f"\n{'='*60}")
    print(f"[User {USER_ID}] STEP {step}: Received matrices from
Resource Manager")
    print(f"{'='*60}")

    T_matrix = np.array(data['T_matrix'])
    E_matrix = np.array(data['E_matrix'])

    #get user 1's row (index 0)
    user_times = T_matrix[0]
    user_expenses = E_matrix[0]

    #use stored allocation instead of recalculating
    if step == 1:
        allocation = STEP1_ALLOCATION
    else:
        allocation = STEP2_ALLOCATION

    if allocation is None:
        print(f"[User {USER_ID}] ERROR: No allocation stored for Step
{step}")
```

```
        return jsonify({'error': f'No allocation stored for Step
{step}'}), 400

    resources_used = [j for j in range(NUM_RESOURCES) if allocation[j]
> 0]

    if not resources_used:
        print(f"[User {USER_ID}] ERROR: No resources allocated!")
        return jsonify({'error': 'No resources allocated'}), 400

    actual_max_time = max([user_times[j] for j in resources_used])
    actual_expense = sum([user_expenses[j] for j in resources_used])

    denominator = W_TIME * actual_max_time + W_EXPENSE * actual_expense
    if denominator == 0:
        actual_utility = float('inf')
        print(f"[User {USER_ID}] WARNING: Utility is infinite (zero
cost/time)")
    else:
        actual_utility = 1 / denominator

    if step == 1:
        STEP1_ACTUAL_UTILITY = actual_utility

    print(f"[User {USER_ID}] STEP {step} Actual results (with resource
sharing):")
    print(f"  Allocation used: {allocation.tolist()}")
    print(f"  Resources used: {resources_used}")
    print(f"  Actual max time: {actual_max_time:.2f}s")
    print(f"  Actual expense: {actual_expense:.2f}€")
    print(f"  Actual utility: {actual_utility:.6f}")

    #receive t-hat-new for step 2
    if 'T_HAT_NEW' in data and step == 1:
        T_HAT_NEW = np.array(data['T_HAT_NEW'])
        STEP2_EXEC_TIMES = T_HAT_NEW[0]

        print(f"\n[User {USER_ID}] Received updated execution times for
STEP 2:")
        print(f"  Original T-hat:     {EXEC_TIMES}")
        print(f"  Updated T-hat-new: {STEP2_EXEC_TIMES}")
        print(f"  Difference:     {STEP2_EXEC_TIMES - EXEC_TIMES}")
```

```
        print(f"\n[User {USER_ID}] Ready for Step 2! Call
/calculate_step2")

    print(f"{'='*60}\n")

    return jsonify({
        'user_id': USER_ID,
        'step': step,
        'status': 'updated',
        'actual_utility': actual_utility if actual_utility !=
float('inf') else 'infinity',
        'actual_max_time': actual_max_time,
        'actual_expense': actual_expense,
        'step2_ready': STEP2_EXEC_TIMES is not None
    })


@app.route('/reset', methods=['POST'])
def reset():
    #reset to step 1 (clear step 2 data)
    global STEP2_EXEC_TIMES, CURRENT_STEP, STEP1_ACTUAL_UTILITY
    global STEP1_EXPECTED_UTILITY, STEP1_ALLOCATION, STEP2_ALLOCATION

    STEP2_EXEC_TIMES = None
    CURRENT_STEP = 1
    STEP1_ACTUAL_UTILITY = None
    STEP1_EXPECTED_UTILITY = None
    STEP1_ALLOCATION = None
    STEP2_ALLOCATION = None

    print(f"[User {USER_ID}] Reset to Step 1")

    return jsonify({
        'user_id': USER_ID,
        'status': 'reset',
        'message': 'Reset to Step 1. Ready for new game.'
    })


if __name__ == '__main__':
    print(f"\n{'='*60}")
    print(f"USER {USER_ID} FLASK APP STARTING")
    print(f"{'='*60}")
```

```
    print(f"Parameters:")
    print(f"  Student ID last digits: 47")
    print(f"  Subtasks: {NUM_SUBTASKS}")
    print(f"  Time weight: {W_TIME}")
    print(f"  Expense weight: {W_EXPENSE}")
    print(f"  Max time: {MAX_TIME}s")
    print(f"  Max expense: {MAX_EXPENSE}€")
    print(f"\nEndpoints:")
    print(f"  GET  /              - Health check")
    print(f"  POST /calculate     - STEP 1: Calculate with original
T-hat")
    print(f"  POST /calculate_step2 - STEP 2: Calculate with
T-hat-new")
    print(f"  POST /update        - Receive matrices from Resource
Manager")
    print(f"  POST /reset         - Reset to Step 1")
    print(f"{'='*60}\n")

    app.run(host='0.0.0.0', port=5000, debug=True)
```

user2_app:

```
from flask import Flask, jsonify, request
import numpy as np
from itertools import product
import boto3
import json

app = Flask(__name__)

#user 2 parameters (student id: 47)
USER_ID = 2
NUM_SUBTASKS = 3
W_TIME = 0.48
W_EXPENSE = 0.52
MAX_TIME = 300
MAX_EXPENSE = 30

PRICES = np.array([1, 1.2, 1.5, 1.8, 2])
EXEC_TIMES = np.array([5, 4.2, 3.6, 3, 2.8])
NUM_RESOURCES = 5

#sqs configuration
```

```
SQS_QUEUE_URL =
'https://sqs.us-east-1.amazonaws.com/519661830216/SubmissionQueue'
sqs = boto3.client('sqs', region_name='us-east-1')

#step 2 tracking variables
STEP2_EXEC_TIMES = None
CURRENT_STEP = 1
STEP1_EXPECTED_UTILITY = None
STEP1_ACTUAL_UTILITY = None
STEP1_ALLOCATION = None
STEP2_ALLOCATION = None


def calculate_utility(allocation, exec_times, prices, w_t, w_e):
    #calculate utility for given allocation
    times_for_allocated = [exec_times[j] for j in range(NUM_RESOURCES)
                           if allocation[j] > 0]
    if not times_for_allocated:
        return 0

    max_time = max(times_for_allocated)
    total_expense = sum(allocation[j] * exec_times[j] * prices[j]
                        for j in range(NUM_RESOURCES))

    denominator = w_t * max_time + w_e * total_expense
    return 1 / denominator if denominator > 0 else 0


def find_optimal_allocation():
    #find optimal allocation using original execution times (step 1)
    best_utility = 0
    best_allocation = None
    best_details = None

    for resources in product(range(NUM_RESOURCES),
repeat=NUM_SUBTASKS):
        if len(set(resources)) != NUM_SUBTASKS:
            continue

        allocation = np.zeros(NUM_RESOURCES, dtype=int)
        for r in resources:
            allocation[r] += 1
```

```
        times = [EXEC_TIMES[j] for j in range(NUM_RESOURCES)
                if allocation[j] > 0]
        max_time = max(times)
        total_expense = sum(allocation[j] * EXEC_TIMES[j] * PRICES[j]
                            for j in range(NUM_RESOURCES))

        if max_time > MAX_TIME or total_expense > MAX_EXPENSE:
            continue

        utility = calculate_utility(allocation, EXEC_TIMES, PRICES,
                                    W_TIME, W_EXPENSE)

        if utility > best_utility:
            best_utility = utility
            best_allocation = allocation
            best_details = {
                'max_time': max_time,
                'total_expense': total_expense,
                'utility': utility
            }

    return best_allocation, best_details


def find_optimal_allocation_step2(exec_times_step2):
    #find optimal allocation using updated execution times from step 2
    best_utility = 0
    best_allocation = None
    best_details = None

    for resources in product(range(NUM_RESOURCES),
repeat=NUM_SUBTASKS):
        if len(set(resources)) != NUM_SUBTASKS:
            continue

        allocation = np.zeros(NUM_RESOURCES, dtype=int)
        for r in resources:
            allocation[r] += 1

        times = [exec_times_step2[j] for j in range(NUM_RESOURCES)
                if allocation[j] > 0]
        max_time = max(times)
```

```
        total_expense = sum(allocation[j] * exec_times_step2[j] *
PRICES[j]
                            for j in range(NUM_RESOURCES))

        if max_time > MAX_TIME or total_expense > MAX_EXPENSE:
            continue

        utility = calculate_utility(allocation, exec_times_step2,
PRICES,
                                    W_TIME, W_EXPENSE)

        if utility > best_utility:
            best_utility = utility
            best_allocation = allocation
            best_details = {
                'max_time': max_time,
                'total_expense': total_expense,
                'utility': utility
            }

    return best_allocation, best_details


@app.route('/')
def home():
    #health check endpoint
    return jsonify({
        'status': 'running',
        'user_id': USER_ID,
        'message': 'User 2 Flask App - CS452 Project',
        'current_step': CURRENT_STEP,
        'step2_ready': STEP2_EXEC_TIMES is not None
    })


@app.route('/calculate', methods=['GET', 'POST'])
def calculate_allocation():
    #step 1: calculate optimal allocation using original t-hat
    global STEP1_EXPECTED_UTILITY, STEP1_ALLOCATION, CURRENT_STEP

    CURRENT_STEP = 1

    print(f"\n{'='*60}")
```

```
    print(f"[User {USER_ID}] STEP 1: Starting optimization
calculation...")
    print(f"{'='*60}")
    print(f"Using ORIGINAL execution times: {EXEC_TIMES}")

    allocation, details = find_optimal_allocation()

    if allocation is None:
        print(f"[User {USER_ID}] ERROR: No feasible allocation found!")
        return jsonify({'error': 'No feasible allocation found'}), 400

    STEP1_EXPECTED_UTILITY = details['utility']
    STEP1_ALLOCATION = allocation

    print(f"[User {USER_ID}] Step 1 optimal allocation found:")
    print(f"  Allocation vector: {allocation.tolist()}")
    print(f"  Expected utility: {details['utility']:.6f}")
    print(f"  Expected max time: {details['max_time']:.2f}s")
    print(f"  Expected expense: {details['total_expense']:.2f}€")

    message = {
        'user_id': USER_ID,
        'step': 1,
        'allocation_vector': allocation.tolist(),
        'expected_utility': details['utility'],
        'expected_max_time': details['max_time'],
        'expected_expense': details['total_expense']
    }

    try:
        response = sqs.send_message(
            QueueUrl=SQS_QUEUE_URL,
            MessageBody=json.dumps(message)
        )
        print(f"[User {USER_ID}] Step 1 allocation sent to SQS")
        print(f"{'='*60}\n")

        return jsonify({
            'user_id': USER_ID,
            'step': 1,
            'allocation_vector': allocation.tolist(),
            'expected_utility': details['utility'],
            'expected_max_time': details['max_time'],
```

```
            'expected_expense': details['total_expense'],
            'message_id': response['MessageId'],
            'status': 'submitted_to_queue'
        })

    except Exception as e:
        print(f"[User {USER_ID}] Error sending to SQS: {e}")
        return jsonify({'error': str(e)}), 500


@app.route('/calculate_step2', methods=['GET', 'POST'])
def calculate_step2():
    #step 2: recalculate allocation using updated t-hat-new
    global CURRENT_STEP, STEP2_EXEC_TIMES, STEP2_ALLOCATION

    if STEP2_EXEC_TIMES is None:
        return jsonify({
            'error': 'Step 2 not ready. Must complete Step 1 first to
receive T-hat-new.'
        }), 400

    CURRENT_STEP = 2

    print(f"\n{'='*60}")
    print(f"[User {USER_ID}] STEP 2: Recalculating with UPDATED
execution times...")
    print(f"{'='*60}")
    print(f"Original T-hat:  {EXEC_TIMES}")
    print(f"Updated T-hat-new: {STEP2_EXEC_TIMES}")
    print(f"Change:      {STEP2_EXEC_TIMES - EXEC_TIMES}")

    allocation, details =
find_optimal_allocation_step2(STEP2_EXEC_TIMES)

    if allocation is None:
        print(f"[User {USER_ID}] ERROR: No feasible allocation found in
Step 2!")
        return jsonify({'error': 'No feasible allocation found'}), 400

    STEP2_ALLOCATION = allocation

    print(f"\n[User {USER_ID}] Step 2 optimal allocation found:")
    print(f"  Allocation vector: {allocation.tolist()}")
```

```
    print(f"  Expected utility: {details['utility']:.6f}")
    print(f"  Expected max time: {details['max_time']:.2f}s")
    print(f"  Expected expense: {details['total_expense']:.2f}€")

    if STEP1_ALLOCATION is not None:
        print(f"\n  COMPARISON WITH STEP 1:")
        print(f"  Step 1 allocation: {STEP1_ALLOCATION.tolist()}")
        print(f"  Step 2 allocation: {allocation.tolist()}")
        print(f"  Changed? {not np.array_equal(STEP1_ALLOCATION,
allocation)}")

    message = {
        'user_id': USER_ID,
        'step': 2,
        'allocation_vector': allocation.tolist(),
        'expected_utility': details['utility'],
        'expected_max_time': details['max_time'],
        'expected_expense': details['total_expense']
    }

    try:
        response = sqs.send_message(
            QueueUrl=SQS_QUEUE_URL,
            MessageBody=json.dumps(message)
        )
        print(f"[User {USER_ID}] Step 2 allocation sent to SQS")
        print(f"{'='*60}\n")

        return jsonify({
            'user_id': USER_ID,
            'step': 2,
            'allocation_vector': allocation.tolist(),
            'expected_utility': details['utility'],
            'expected_max_time': details['max_time'],
            'expected_expense': details['total_expense'],
            'message_id': response['MessageId'],
            'status': 'submitted_to_queue'
        })

    except Exception as e:
        print(f"[User {USER_ID}] Error sending to SQS: {e}")
        return jsonify({'error': str(e)}), 500
```

```
@app.route('/update', methods=['POST'])
def receive_update():
    #receive updated t and e matrices from resource manager
    #uses stored allocations instead of recalculating
    global STEP2_EXEC_TIMES, STEP1_ACTUAL_UTILITY

    data = request.json
    step = data.get('step', 1)

    print(f"\n{'='*60}")
    print(f"[User {USER_ID}] STEP {step}: Received matrices from
Resource Manager")
    print(f"{'='*60}")

    T_matrix = np.array(data['T_matrix'])
    E_matrix = np.array(data['E_matrix'])

    #get user 2's row (index 1)
    user_times = T_matrix[1]
    user_expenses = E_matrix[1]

    #use stored allocation instead of recalculating
    if step == 1:
        allocation = STEP1_ALLOCATION
    else:
        allocation = STEP2_ALLOCATION

    if allocation is None:
        print(f"[User {USER_ID}] ERROR: No allocation stored for Step
{step}")
        return jsonify({'error': f'No allocation stored for Step
{step}'}), 400

    resources_used = [j for j in range(NUM_RESOURCES) if allocation[j]
> 0]

    if not resources_used:
        print(f"[User {USER_ID}] ERROR: No resources allocated!")
        return jsonify({'error': 'No resources allocated'}), 400

    actual_max_time = max([user_times[j] for j in resources_used])
    actual_expense = sum([user_expenses[j] for j in resources_used])
```

```
    denominator = W_TIME * actual_max_time + W_EXPENSE * actual_expense
    if denominator == 0:
        actual_utility = float('inf')
        print(f"[User {USER_ID}] WARNING: Utility is infinite (zero
cost/time)")
    else:
        actual_utility = 1 / denominator

    if step == 1:
        STEP1_ACTUAL_UTILITY = actual_utility

    print(f"[User {USER_ID}] STEP {step} Actual results (with resource
sharing):")
    print(f"  Allocation used: {allocation.tolist()}")
    print(f"  Resources used: {resources_used}")
    print(f"  Actual max time: {actual_max_time:.2f}s")
    print(f"  Actual expense: {actual_expense:.2f}€")
    print(f"  Actual utility: {actual_utility:.6f}")

    #receive t-hat-new for step 2
    if 'T_HAT_NEW' in data and step == 1:
        T_HAT_NEW = np.array(data['T_HAT_NEW'])
        STEP2_EXEC_TIMES = T_HAT_NEW[1]

        print(f"\n[User {USER_ID}] Received updated execution times for
STEP 2:")
        print(f"  Original T-hat:     {EXEC_TIMES}")
        print(f"  Updated T-hat-new: {STEP2_EXEC_TIMES}")
        print(f"  Difference:     {STEP2_EXEC_TIMES - EXEC_TIMES}")
        print(f"\n[User {USER_ID}] Ready for Step 2! Call
/calculate_step2")

    print(f"{'='*60}\n")

    return jsonify({
        'user_id': USER_ID,
        'step': step,
        'status': 'updated',
        'actual_utility': actual_utility if actual_utility !=
float('inf') else 'infinity',
        'actual_max_time': actual_max_time,
        'actual_expense': actual_expense,
```

```
        'step2_ready': STEP2_EXEC_TIMES is not None
    })


@app.route('/reset', methods=['POST'])
def reset():
    #reset to step 1 (clear step 2 data)
    global STEP2_EXEC_TIMES, CURRENT_STEP, STEP1_ACTUAL_UTILITY
    global STEP1_EXPECTED_UTILITY, STEP1_ALLOCATION, STEP2_ALLOCATION

    STEP2_EXEC_TIMES = None
    CURRENT_STEP = 1
    STEP1_ACTUAL_UTILITY = None
    STEP1_EXPECTED_UTILITY = None
    STEP1_ALLOCATION = None
    STEP2_ALLOCATION = None

    print(f"[User {USER_ID}] Reset to Step 1")

    return jsonify({
        'user_id': USER_ID,
        'status': 'reset',
        'message': 'Reset to Step 1. Ready for new game.'
    })


if __name__ == '__main__':
    print(f"\n{'='*60}")
    print(f"USER {USER_ID} FLASK APP STARTING")
    print(f"{'='*60}")
    print(f"Parameters:")
    print(f"  Student ID last digits: 47")
    print(f"  Subtasks: {NUM_SUBTASKS}")
    print(f"  Time weight: {W_TIME}")
    print(f"  Expense weight: {W_EXPENSE}")
    print(f"  Max time: {MAX_TIME}s")
    print(f"  Max expense: {MAX_EXPENSE}€")
    print(f"\nEndpoints:")
    print(f"  GET  /              - Health check")
    print(f"  POST /calculate     - STEP 1: Calculate with original
T-hat")
    print(f"  POST /calculate_step2 - STEP 2: Calculate with
T-hat-new")
```

```
    print(f"  POST /update        - Receive matrices from Resource
Manager")
    print(f"  POST /reset         - Reset to Step 1")
    print(f"{'='*60}\n")

    app.run(host='0.0.0.0', port=5000, debug=True)
```

user3_app:

```
from flask import Flask, jsonify, request
import numpy as np
from itertools import product
import boto3
import json

app = Flask(__name__)

#user 3 parameters (student id: 47)
USER_ID = 3
NUM_SUBTASKS = 4
W_TIME = 0.49
W_EXPENSE = 0.51
MAX_TIME = 1200
MAX_EXPENSE = 40

PRICES = np.array([1, 1.2, 1.5, 1.8, 2])
EXEC_TIMES = np.array([4, 3.5, 3.2, 2.8, 2.4])
NUM_RESOURCES = 5

#sqs configuration
SQS_QUEUE_URL =
'https://sqs.us-east-1.amazonaws.com/519661830216/SubmissionQueue'
sqs = boto3.client('sqs', region_name='us-east-1')

#step 2 tracking variables
STEP2_EXEC_TIMES = None
CURRENT_STEP = 1
STEP1_EXPECTED_UTILITY = None
STEP1_ACTUAL_UTILITY = None
STEP1_ALLOCATION = None
STEP2_ALLOCATION = None


def calculate_utility(allocation, exec_times, prices, w_t, w_e):
```

```
    #calculate utility for given allocation
    times_for_allocated = [exec_times[j] for j in range(NUM_RESOURCES)
                           if allocation[j] > 0]
    if not times_for_allocated:
        return 0

    max_time = max(times_for_allocated)
    total_expense = sum(allocation[j] * exec_times[j] * prices[j]
                        for j in range(NUM_RESOURCES))

    denominator = w_t * max_time + w_e * total_expense
    return 1 / denominator if denominator > 0 else 0


def find_optimal_allocation():
    #find optimal allocation using original execution times (step 1)
    best_utility = 0
    best_allocation = None
    best_details = None

    for resources in product(range(NUM_RESOURCES),
repeat=NUM_SUBTASKS):
        if len(set(resources)) != NUM_SUBTASKS:
            continue

        allocation = np.zeros(NUM_RESOURCES, dtype=int)
        for r in resources:
            allocation[r] += 1

        times = [EXEC_TIMES[j] for j in range(NUM_RESOURCES)
                 if allocation[j] > 0]
        max_time = max(times)
        total_expense = sum(allocation[j] * EXEC_TIMES[j] * PRICES[j]
                            for j in range(NUM_RESOURCES))

        if max_time > MAX_TIME or total_expense > MAX_EXPENSE:
            continue

        utility = calculate_utility(allocation, EXEC_TIMES, PRICES,
                                    W_TIME, W_EXPENSE)

        if utility > best_utility:
            best_utility = utility
```

```
            best_allocation = allocation
            best_details = {
                'max_time': max_time,
                'total_expense': total_expense,
                'utility': utility
            }

    return best_allocation, best_details


def find_optimal_allocation_step2(exec_times_step2):
    #find optimal allocation using updated execution times from step 2
    best_utility = 0
    best_allocation = None
    best_details = None

    for resources in product(range(NUM_RESOURCES),
repeat=NUM_SUBTASKS):
        if len(set(resources)) != NUM_SUBTASKS:
            continue

        allocation = np.zeros(NUM_RESOURCES, dtype=int)
        for r in resources:
            allocation[r] += 1

        times = [exec_times_step2[j] for j in range(NUM_RESOURCES)
                if allocation[j] > 0]
        max_time = max(times)
        total_expense = sum(allocation[j] * exec_times_step2[j] *
PRICES[j]
                            for j in range(NUM_RESOURCES))

        if max_time > MAX_TIME or total_expense > MAX_EXPENSE:
            continue

        utility = calculate_utility(allocation, exec_times_step2,
PRICES,
                                    W_TIME, W_EXPENSE)

        if utility > best_utility:
            best_utility = utility
            best_allocation = allocation
            best_details = {
```

```
                'max_time': max_time,
                'total_expense': total_expense,
                'utility': utility
            }

    return best_allocation, best_details


@app.route('/')
def home():
    #health check endpoint
    return jsonify({
        'status': 'running',
        'user_id': USER_ID,
        'message': 'User 3 Flask App - CS452 Project',
        'current_step': CURRENT_STEP,
        'step2_ready': STEP2_EXEC_TIMES is not None
    })


@app.route('/calculate', methods=['GET', 'POST'])
def calculate_allocation():
    #step 1: calculate optimal allocation using original t-hat
    global STEP1_EXPECTED_UTILITY, STEP1_ALLOCATION, CURRENT_STEP

    CURRENT_STEP = 1

    print(f"\n{'='*60}")
    print(f"[User {USER_ID}] STEP 1: Starting optimization
calculation...")
    print(f"{'='*60}")
    print(f"Using ORIGINAL execution times: {EXEC_TIMES}")

    allocation, details = find_optimal_allocation()

    if allocation is None:
        print(f"[User {USER_ID}] ERROR: No feasible allocation found!")
        return jsonify({'error': 'No feasible allocation found'}), 400

    STEP1_EXPECTED_UTILITY = details['utility']
    STEP1_ALLOCATION = allocation

    print(f"[User {USER_ID}] Step 1 optimal allocation found:")
```

```
    print(f"  Allocation vector: {allocation.tolist()}")
    print(f"  Expected utility: {details['utility']:.6f}")
    print(f"  Expected max time: {details['max_time']:.2f}s")
    print(f"  Expected expense: {details['total_expense']:.2f}€")

    message = {
        'user_id': USER_ID,
        'step': 1,
        'allocation_vector': allocation.tolist(),
        'expected_utility': details['utility'],
        'expected_max_time': details['max_time'],
        'expected_expense': details['total_expense']
    }

    try:
        response = sqs.send_message(
            QueueUrl=SQS_QUEUE_URL,
            MessageBody=json.dumps(message)
        )
        print(f"[User {USER_ID}] Step 1 allocation sent to SQS")
        print(f"{'='*60}\n")

        return jsonify({
            'user_id': USER_ID,
            'step': 1,
            'allocation_vector': allocation.tolist(),
            'expected_utility': details['utility'],
            'expected_max_time': details['max_time'],
            'expected_expense': details['total_expense'],
            'message_id': response['MessageId'],
            'status': 'submitted_to_queue'
        })

    except Exception as e:
        print(f"[User {USER_ID}] Error sending to SQS: {e}")
        return jsonify({'error': str(e)}), 500


@app.route('/calculate_step2', methods=['GET', 'POST'])
def calculate_step2():
    #step 2: recalculate allocation using updated t-hat-new
    global CURRENT_STEP, STEP2_EXEC_TIMES, STEP2_ALLOCATION
```

```
    if STEP2_EXEC_TIMES is None:
        return jsonify({
            'error': 'Step 2 not ready. Must complete Step 1 first to
receive T-hat-new.'
        }), 400

    CURRENT_STEP = 2

    print(f"\n{'='*60}")
    print(f"[User {USER_ID}] STEP 2: Recalculating with UPDATED
execution times...")
    print(f"{'='*60}")
    print(f"Original T-hat:  {EXEC_TIMES}")
    print(f"Updated T-hat-new: {STEP2_EXEC_TIMES}")
    print(f"Change:      {STEP2_EXEC_TIMES - EXEC_TIMES}")

    allocation, details =
find_optimal_allocation_step2(STEP2_EXEC_TIMES)

    if allocation is None:
        print(f"[User {USER_ID}] ERROR: No feasible allocation found in
Step 2!")
        return jsonify({'error': 'No feasible allocation found'}), 400

    STEP2_ALLOCATION = allocation

    print(f"\n[User {USER_ID}] Step 2 optimal allocation found:")
    print(f"  Allocation vector: {allocation.tolist()}")
    print(f"  Expected utility: {details['utility']:.6f}")
    print(f"  Expected max time: {details['max_time']:.2f}s")
    print(f"  Expected expense: {details['total_expense']:.2f}€")

    if STEP1_ALLOCATION is not None:
        print(f"\n  COMPARISON WITH STEP 1:")
        print(f"  Step 1 allocation: {STEP1_ALLOCATION.tolist()}")
        print(f"  Step 2 allocation: {allocation.tolist()}")
        print(f"  Changed? {not np.array_equal(STEP1_ALLOCATION,
allocation)}")

    message = {
        'user_id': USER_ID,
        'step': 2,
        'allocation_vector': allocation.tolist(),
```

```
        'expected_utility': details['utility'],
        'expected_max_time': details['max_time'],
        'expected_expense': details['total_expense']
    }

    try:
        response = sqs.send_message(
            QueueUrl=SQS_QUEUE_URL,
            MessageBody=json.dumps(message)
        )
        print(f"[User {USER_ID}] Step 2 allocation sent to SQS")
        print(f"{'='*60}\n")

        return jsonify({
            'user_id': USER_ID,
            'step': 2,
            'allocation_vector': allocation.tolist(),
            'expected_utility': details['utility'],
            'expected_max_time': details['max_time'],
            'expected_expense': details['total_expense'],
            'message_id': response['MessageId'],
            'status': 'submitted_to_queue'
        })

    except Exception as e:
        print(f"[User {USER_ID}] Error sending to SQS: {e}")
        return jsonify({'error': str(e)}), 500


@app.route('/update', methods=['POST'])
def receive_update():
    #receive updated t and e matrices from resource manager
    #uses stored allocations instead of recalculating
    global STEP2_EXEC_TIMES, STEP1_ACTUAL_UTILITY

    data = request.json
    step = data.get('step', 1)

    print(f"\n{'='*60}")
    print(f"[User {USER_ID}] STEP {step}: Received matrices from
Resource Manager")
    print(f"{'='*60}")
```

```
    T_matrix = np.array(data['T_matrix'])
    E_matrix = np.array(data['E_matrix'])

    #get user 3's row (index 2)
    user_times = T_matrix[2]
    user_expenses = E_matrix[2]

    #use stored allocation instead of recalculating
    if step == 1:
        allocation = STEP1_ALLOCATION
    else:
        allocation = STEP2_ALLOCATION

    if allocation is None:
        print(f"[User {USER_ID}] ERROR: No allocation stored for Step
{step}")
        return jsonify({'error': f'No allocation stored for Step
{step}'}), 400

    resources_used = [j for j in range(NUM_RESOURCES) if allocation[j]
> 0]

    if not resources_used:
        print(f"[User {USER_ID}] ERROR: No resources allocated!")
        return jsonify({'error': 'No resources allocated'}), 400

    actual_max_time = max([user_times[j] for j in resources_used])
    actual_expense = sum([user_expenses[j] for j in resources_used])

    denominator = W_TIME * actual_max_time + W_EXPENSE * actual_expense
    if denominator == 0:
        actual_utility = float('inf')
        print(f"[User {USER_ID}] WARNING: Utility is infinite (zero
cost/time)")
    else:
        actual_utility = 1 / denominator

    if step == 1:
        STEP1_ACTUAL_UTILITY = actual_utility

    print(f"[User {USER_ID}] STEP {step} Actual results (with resource
sharing):")
    print(f"  Allocation used: {allocation.tolist()}")
```

```
    print(f"  Resources used: {resources_used}")
    print(f"  Actual max time: {actual_max_time:.2f}s")
    print(f"  Actual expense: {actual_expense:.2f}€")
    print(f"  Actual utility: {actual_utility:.6f}")

    #receive t-hat-new for step 2
    if 'T_HAT_NEW' in data and step == 1:
        T_HAT_NEW = np.array(data['T_HAT_NEW'])
        STEP2_EXEC_TIMES = T_HAT_NEW[2]

        print(f"\n[User {USER_ID}] Received updated execution times for
STEP 2:")
        print(f"  Original T-hat:     {EXEC_TIMES}")
        print(f"  Updated T-hat-new: {STEP2_EXEC_TIMES}")
        print(f"  Difference:     {STEP2_EXEC_TIMES - EXEC_TIMES}")
        print(f"\n[User {USER_ID}] Ready for Step 2! Call
/calculate_step2")

    print(f"{'='*60}\n")

    return jsonify({
        'user_id': USER_ID,
        'step': step,
        'status': 'updated',
        'actual_utility': actual_utility if actual_utility !=
float('inf') else 'infinity',
        'actual_max_time': actual_max_time,
        'actual_expense': actual_expense,
        'step2_ready': STEP2_EXEC_TIMES is not None
    })


@app.route('/reset', methods=['POST'])
def reset():
    #reset to step 1 (clear step 2 data)
    global STEP2_EXEC_TIMES, CURRENT_STEP, STEP1_ACTUAL_UTILITY
    global STEP1_EXPECTED_UTILITY, STEP1_ALLOCATION, STEP2_ALLOCATION

    STEP2_EXEC_TIMES = None
    CURRENT_STEP = 1
    STEP1_ACTUAL_UTILITY = None
    STEP1_EXPECTED_UTILITY = None
    STEP1_ALLOCATION = None
```

```
    STEP2_ALLOCATION = None

    print(f"[User {USER_ID}] Reset to Step 1")

    return jsonify({
        'user_id': USER_ID,
        'status': 'reset',
        'message': 'Reset to Step 1. Ready for new game.'
    })


if __name__ == '__main__':
    print(f"\n{'='*60}")
    print(f"USER {USER_ID} FLASK APP STARTING")
    print(f"{'='*60}")
    print(f"Parameters:")
    print(f"  Student ID last digits: 47")
    print(f"  Subtasks: {NUM_SUBTASKS}")
    print(f"  Time weight: {W_TIME}")
    print(f"  Expense weight: {W_EXPENSE}")
    print(f"  Max time: {MAX_TIME}s")
    print(f"  Max expense: {MAX_EXPENSE}€")
    print(f"\nEndpoints:")
    print(f"  GET  /              - Health check")
    print(f"  POST /calculate     - STEP 1: Calculate with original
T-hat")
    print(f"  POST /calculate_step2 - STEP 2: Calculate with
T-hat-new")
    print(f"  POST /update        - Receive matrices from Resource
Manager")
    print(f"  POST /reset         - Reset to Step 1")
    print(f"{'='*60}\n")

    app.run(host='0.0.0.0', port=5000, debug=True)
```